

Operating Systems – 67808

Exercise 2 - MapReduce - Multi-threaded Programming

Note: **This exercise takes a lot of time. Start early!**

High Level Overview

Performance is the major motivation for multi-threaded programming. Multiple processors can execute multiple threads at the same time and perform the same amount of computation in less time than it will take a single processor.

Two challenges complicate multi-threaded programming:

1. In many cases it is difficult to split the big task into small parts that can run in parallel.
2. Running in multiple threads requires synchronization and communication between threads. This introduces an overhead which without careful design can increase the total runtime significantly.

Over the years, several designs were proposed in order to solve these challenges. In this exercise we will implement one of these designs, called [MapReduce](#).

MapReduce is used to parallelize tasks with a specific structure. Such tasks are defined by two functions, *map* and *reduce*, used as follows:

1. The input is given as a sequence of elements.
2. (Map phase) The *map* function is applied to each input element, producing a sequence of intermediary elements **for each thread**.
3. (Sort phase) Each thread sorts its own intermediate vector (by key).
4. (Shuffle phase) Only one thread creates, for each key, the list of values associated with the keys, taking all values from all threads.

5. (Reduce phase) The *reduce* function is applied to each of the sorted sequences of intermediary elements, producing a sequence of output elements.
6. The output is a sorted concatenation of all sequences of output elements.

Example (From Tirgul): counting character frequency in strings

Explanation	Example for 2 threads
The input is a sequence of strings.	Input: "This string is full of characters" "Multithreading is awesome" "race conditions are bad"
(Map phase) For each string, we count how many times each character appears and then produce a sequence of the results.	Thread 0 (got strings #1 and #3): <T, 1>, <h, 2>, ..., <c, 2>, ..., <s, 4> Thread 1 (got string #2): <M, 1>, <u, 1>, ..., <e, 2>, ..., <o, 1> The internal order does not matter!
(Sort phase) Each thread sorts the list by the characters (in this case: space first, then upper-case letters, then lower-case)	Thread 0 (strings #1 and #3): <" ", 5>, <" ", 2> <T, 1>, <a, 2>, <a, 3>, <b, 1>, <c, 2>, <c, 2>, ..., <u, 1> Thread 1 (string #2): <" ", 3>, <M, 1>, <a, 2>, <d, 1>, ..., <u, 1> Here the internal order does matter!
(Shuffle phase) Only one thread group together same keys into a list of pairs	One list of lists (of pairs): <<" ", 5>, <" ", 2>, <" ", 3>> <<M, 1>> <<T, 1>> <<a, 2>, <a, 3>, <a, 2>> ... <<u, 1>, <u, 1>>
(Reduce phase) Each thread takes pair lists (each list of pairs correspond to a character)	Thread 0:

	$\langle \langle " ", 5 \rangle, \langle " ", 2 \rangle, \langle " ", 3 \rangle \rangle \rightarrow \langle " ", 10 \rangle$ $\langle \langle a, 2 \rangle, \langle a, 3 \rangle, \langle a, 2 \rangle \rangle \rightarrow \langle a, 7 \rangle$... (and more) Thread 1: $\langle \langle M, 1 \rangle \rangle \rightarrow \langle M, 1 \rangle$ $\langle \langle T, 1 \rangle \rangle \rightarrow \langle T, 1 \rangle$... (and more)
--	--

Attention: the order in which the threads take/process the pairs or list of pairs is **not necessarily reproducible!** In other words, if you run it once again, the same thread might have different processed elements.

Design

The implementation of this design can be split into two parts:

1. Implementing the functions *map* and *reduce*. The functions implementation will be different for every task. We call this part the client.
2. Implementing everything else – the partition into phases, distribution of work between threads, synchronization etc. This will be identical for different tasks. We call this part the framework.

Using this split, we can code the framework once and then use it for different scenarios, each has its own client implementation.

In other words, after you have written the code for the framework, the user uses it as a black box, solely writing their own map and reduce functions.

Constructing the framework is the main goal of this exercise and you will have to **implement the framework**. In the next sections we will break this goal down into subgoals and provide a more detailed design for you to implement.

Client Overview

This is the part you **do not** implement, but you should be aware of (and also, you need it for test writing).

The client contains two main functions: map and reduce.

Look at the example Client!

Since the elements after the map and reduce function should have linear order, every element must have a key that allows us to compare elements and sort them. For this reason, each element is given as a pair (key, value).

We have three types of elements, each having its own key type and value type:

1. **Input elements** – we denote their key type $k1$ and value type $v1$.
2. **Intermediary elements** – we denote their key type $k2$ and value type $v2$.
3. **Output elements** – we denote their key type $k3$ and value type $v3$.

The map function receives a key of type $k1$ and a value of type $v1$ as input and produces pairs of $(k2, v2)$.

The framework sort/shuffle phases sort the intermediary elements that have been created by the map function according to their keys and then create new sequences such that *reduce* will run exactly once for each $k2$.

The reduce function receives a sequence of pairs $(k2, v2)$ as input, where all keys are identical, and produces pairs of $(k3, v3)$.

A header *MapReduceClient.h* and a sample client are provided with this exercise.

SampleClient.cpp implements the counting characters frequency in strings example from the Tirgul. An implementation of a client header *MapReduceClient.h* contains the following:

1. Key/Value classes that inherit¹ from $K1, K2, K3$ and $V1, V2, V3$ including a $<$ operator for the keys, to enable comparison between different elements.
2. The map function with the signature:

```
void map(const std::shared_ptr<K1> key, const std::shared_ptr<V1>
value, MapContext &context) const
```

¹ Inheritance (a public inheritance in C++) of class B from class A means that a pointer/reference to A might point/refer to an object of type B (Polymorphism). Also, it means that B inherits (gets) A's public (and protected) members (methods and fields).

If A has a method called *foo*, it may declare it virtual. This way, B can override the method. Calling to *foo* will determine which *foo* to call (A's A::foo or B's B::foo) according to the object class, in running time.

This function will produce intermediate pairs and will add them to the framework databases using the framework function `context.addIntermediate(k2, v2)`.

Look at the example Client!

The context class is used by the map function to report which elements should be added to the intermediate vector.

3. The `reduce` function with the signature:

```
void reduce(const IntermediateVec &pairs, ReduceContext &context)
const
```

Where `IntermediateVec` is of type `std::vector<std::pair<K2*, V2*>>` (used for abbreviation).

All pairs in the vector are expected to have the same key (but not necessarily the same instances of `K2`).

This function will produce output pairs and will add them to the framework databases using the framework function `context.addOutput(k3, v3)`.

Look at the example Client!

The context argument is used to receive information from `reduce` about what should be added to the output vector.

Pay attention that the `map` and `reduce` functions are called within the framework and the input to these functions is passed by the framework.

Framework Interface Overview

The framework will support running MapReduce operations as an asynchronous **job**, together with the ability to query the current state of a job while it is running.

These are the classes used in our framework:

1. `MapContext` - a class you should implement according to your own discretion. It should implement the method `addIntermediate(k2, v2)`. A reference to a `MapContext` is sent to `map`.

Hint: you may have more than one context of this type for a single job.

2. **ReduceContext** - a class you should implement according to your own discretion. It should implement the method `addOutput(k3, v3)`.

A reference to a **ReduceContext** is sent to `reduce`.

Hint: you may have only one context of this type for a single job.

3. **MapReduceState** - a class that quantizes the state of a job, by holding:

- **State** - an enum (0-Undefined, 1-Map, 2-Shuffle, 3-Reduce)

The sort phase itself does not have a state (since some threads can be in *sort*, while others are still in *map*. Additionally, it's unclear how to measure progress of sorting).

- **double percentage** - job progress of the current phase (see the general remarks about how to measure percentage).

4. **MapReduceJob** - a class that represents a single MapReduce job.

The user is allowed to have multiple jobs running at the same time.

Upon construction of an object of type **MapReduceJob**, the MapReduce operation begins immediately.

The user can check the current progress (get a **MapReduceState** object), wait until the job is done, and wait for its output.

An object of type **MapReduceJob** can not be destroyed before the MapReduce is done.

The MapReduceJob Class

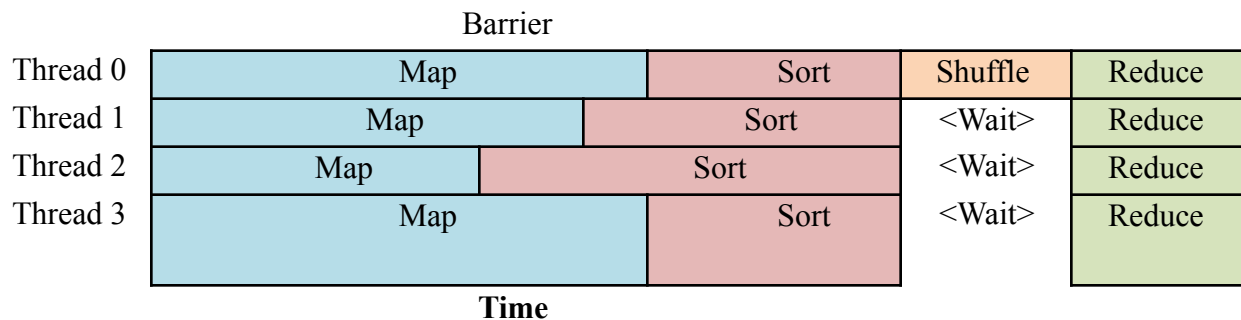
The framework supports the following API:

<code>MapReduceJob(const MapReduceClient &client, const InputVec &inputVec, int multiThreadLevel);</code>	Constructor gets a MapReduceClient object, the input vector, and the number of threads to use. Create threads using the C++ standard threads library , as seen in class.
<code>~MapReduceJob();</code>	A blocking destructor: waits until the MapReduce job is done.
<code>MapReduceState getState(void) const</code>	A nonblocking function: returns the current state of the MapReduce job.
<code>bool isDone(void) const</code>	A nonblocking function: returns a boolean

	indicating whether the MapReduce job has finished.
<code>void wait(void)</code>	A blocking function: waits until the MapReduce job is done. If it's already done, the function returns immediately. Hint – you should use the join method. Hint #2 - Do not call <code>join()</code> twice on a thread. That may lead to unexpected behavior. Hint #3 - This method should be thread-safe. Test a call to <code>wait()</code> from two threads.
<code>OutputVec getOutput(void)</code>	A blocking function: returns a sorted output vector of the MapReduce job, only after it is done. If it's not done, it waits (calls <code>wait</code>). use <code>std::sort</code> to sort, don't worry about ties.

Framework Implementation Details

We will implement a variant of the MapReduce model according to the following diagram:



This diagram shows 4 threads, which is the case for `multiThreadLevel=4`.

In this design all threads except thread 0 run three phases: Map, Sort and Reduce, while thread 0 also runs a special shuffle phase between its Sort and Reduce phases.

In the general case (where `multiThreadLevel=n`):

- Thread 0 runs Map, Sort, Shuffle and then Reduce.
- Threads 1 through n-1 runs Map, Sort and then Reduce.

Map Phase

In this phase each thread reads pairs of $(k1, v1)$ from the input vector and calls the *map* function on each of them. The **map** function in turn will produce $(k2, v2)$ and will call the context's `addIntermediate` function to update the framework databases. We have two synchronization challenges here:

1. Safely split the input between the threads - each thread should “take” one input pair, as long as there are any remaining.
2. Prevent output race conditions – This will be done by separating the outputs. You should create a vector for each thread and then `MapContext::addIntermediate` will just append the new $(k2, v2)$ pair into the calling thread's vector.
 - This solution will solve a race condition between the different Map threads since every Map thread will work with a different vector.
 - The Shuffle thread will combine those vectors into a single data structure.

By the end of this phase, we have `#multiThreadLevel` vectors of intermediate pairs $(k2, v2)$ pairs) and all elements of the input vector have been processed.

Sort Phase

Immediately after the Map phase each thread will sort **its own** intermediate vector sorted according to the keys within. Since every thread has its own vector, this phase needs no special synchronization.

The Shuffle phase must only start after all threads finish their sort phases.

Shuffle Phase

Recall that our goal in this phase is to create new sequences of $(k2, v2)$ where in each sequence all keys are identical and all elements with the same key are placed in a single sequence.

Since our intermediary vectors are sorted, we know that all elements with the largest key must be at the back of each vector. Thus, creating the new sequence is simply a matter of popping these elements from the back of each vector and inserting them to a new vector. Now all elements with

the second largest key are at the back of the vectors so we can repeat the process until the intermediary vectors are empty.

This task is difficult to split efficiently across parallel threads, so we use a single thread doing the work (solely in this phase!).

Shuffle thread (thread 0) while all the other threads will wait until the shuffle phase is completed.

When a new vector for some identical key is created, put it in a queue (shared to all threads). Once all intermediary vectors are empty, we are done and we move on to the Reduce phase.

Reduce Phase

The reducing threads will wait for the shuffled vectors to be created by the shuffling thread. Once they wake up, they can pop a vector from the back of the queue and run *reduce* on it (Remember to lock the mutex when necessary).

The *reduce* function in turn will produce $(k3, v3)$ pairs and will call the context's *addOutput* to add them to the framework data structures.

General Remarks

1. Remember that your library should be thread-safe. Introduce protections to prevent mutual exclusions problems (for example, use mutex/condition variables where needed). Yet, think about the performance! For example, using an atomic variable, when possible, is better than using mutex (why?).
2. Use computational lightweight synchronization primitives. For example, instead of a busy-wait loop, consider using a mutex or a condition variable.
3. Implementation of the job state - pay attention that this struct consists of three different variables (stage, processed, total). When you move between stages, you might be thinking that changing them one after the other (3 lines of code) is safe. But if a user calls for state information by the middle, they might get corrupted or inaccurate information. Handle that problem by using an **atomic variable** of 64 bits (`std::atomic<uint64_t>`) - 2 bits represent the state (UNDEFINED, MAP, SHUFFLE, REDUCE), 31 bits represent the total number of keys to process and 31 other bits represent the already processed keys and 31 other bits. Use atomic operations on this variable to prevent possible corruptions.

STAGE	TOTAL TO PROCESS	PROCESSED
2	31	31

4. No need to handle system/standard libraries fails, exceptions, or any memory allocation problems. **You may assume all given input is valid.**
5. The values variables (V1, V2, V3) are actually unused by the framework (they are only used, and “understood”, by the client’s `map` and `reduce` procedures). So... do not touch them! And in particular, **DO NOT compare values pointers to nullptr** - nullptr values pointers are OK.
6. The job state progress percentage is defined as the percentage of (key, value) pairs processed so far, out of all the (key, value) pairs that need to be processed in the current stage.
 - In the map stage the percentage is the number of input vector items processed out of all the input vector items (input vector size).
 - In the shuffle stage the percentage is the number of intermediate **pairs** shuffled out of the number of all the intermediate **pairs**.
(Extraction of an intermediate vector for a key, increases the progress by its number of elements).
 - In the reduce stage the percentage is the number of shuffled **pairs** (key, value) that have been reduced out of the number of all the shuffled **pairs**.
(Processing a vector of “intermediates” pairs, increases the progress by its number of elements).
7. The shuffle phase will start only after all threads have completed the map and sort phase.

Your Assignment and Restrictions

1. In this exercise only you can write in C++20 (compile with “--std=c++20”), no external libraries are permitted.
2. You should write the framework by:
 - Editing the class MapReduceJob’s header file, and implementing a source file.
Use the provided files in Moodle as templates.

- Editing the classes MapContext and ReduceContext - you should write their header files and source files.

Use the provided files in Moodle as templates.

So, you **should** submit MapReduceJob.h, MapContext.h, ReduceContext.h and their source files.

3. You must **NOT** change the client header (MapReduceClient.h) and the helper header (MapReduceKeys.h) and you CAN NOT submit them.
4. You must use the C++ standard library only - no external libraries are allowed. You **should not** use C-binded API, like pthread_t (pthread.h), sem_t (semaphore.h), mutex_t (pthread.h) and others.
 - You are also not allowed to use pipes, user level threads or forks (no need to, anyway).
5. Your code must be Thread-safe, MapReduceJob must work correctly when called from two different threads simultaneously. Think what implications this has on your design.
6. Pay attention to your runtime and complexity. You're building a library: this exercise **is all about performance**. You should still avoid unnecessary copying of data or other preventable performance pains.
7. You must have no memory leaks or violations (check valgrind!).
8. Since you're writing a library, the code you submit must not contain a main function and should not print anything.

Tips

1. **Recall C++ basics:** references, classes, inheritance.
2. **Shared pointers:** we use shared pointers in this exercise. Even if you don't know what they are, these creatures are very friendly, nice and easy to work with.

Instead of:

```
T *ptr = new T(...);
```

You call:

```
std::shared_ptr<T> ptr = std::make_shared<T>(...)
```

Where “...” replaces the call to the constructor.

it works just like a pointer - (*ptr for dereferencing), you can also use the get method to get a raw pointer (ptr.get() returns a C pointer).

Why working with shared pointers? Because if you use them, it's promised you'll have no memory leaks. Shared pointers use the C++ RAII syntax to release memory automatically, when such a pointer is unused - no need to call delete/delete[]!

What should you know? The only restriction you should be aware about is that you must not create circular chains. For example: if class A has a field of a shared pointer to class B, that itself has a shared pointer to class A - your program will stall by the time A's or B's objects are being destroyed (we use std::weak_ptr to solve this issue).

Note: the shared pointer uses a reference counter to know when it's fine to free the allocated memory (just like in Python).
Recall that in the Tirlgul we've seen what happens to a counter reference being shared by multiple threads.
Therefore, the shared pointer implementation uses an atomic counter. You can use it safely in multiple threads but remember: because of the counter, assignment costs!

3. **Atomic Variables and Bitwise Operations:** Have a look at the atomic demo we've provided you in the exercise files (it also demonstrates bit management).
4. Since the keys have only the < operator and not ==, you can check if two keys a,b are identical by checking whether both a < b and b < a are false.
5. Test early, test often: Using the example client to make sure each phase works correctly before heading to the next phase.
6. Go through all the resources provided before you write code and make sure you understand them. It will save you time.
7. The sample client is **not enough**. Make more complicated clients and test with them.
8. Useful classes and functions (It's possible to use them only partially):
 - static_cast<T>
 - std::thread / std::jthread
 - std::mutex
 - std::condition_variable
 - [std::barrier<>](#) (you may assume we compile with C++20)
 - std::atomic<T>
 - std::sort

- `std::counting_semaphore<Nmax>`
 - `std::atomic_flag`
9. Think about what is shared between all threads (class variables), and what's private to each thread (stack variables).
- Remember that mutexes/barriers/cvs should only be initialized as shared classes fields (do not write the following line inside a thread function: “`std::mutex m;`”)

Did You Understand the Exercise?

Here are some questions that you should be able to answer, or at least to deeply think about:

- What parts do you implement? Do you implement the map/reduce functions?
- Why do we need separate intermediate vectors per thread?
- How do I prevent two threads from processing the same input?
- Why do we need the atomic use in state, and how do I implement getState correctly?
- What is shuffle even doing and why does only thread 0 do it?
- How do threads synchronize between phases?
- What's the purpose of MapContext / ReduceContext and what should they contain?
- What progress do I count in each phase?
- Why is the job's destructor blocking?

If you're struggling with one or more of these questions, DO NOT START THE EXERCISE. Instead, we recommend you visit us in Lab Support or Office Hours.

Presubmission and School Solution

Presubmission tests are available on:

```
~os/ex2_presubmit <path>
```

Where <path> can be a path to a zip/tar file, or a path to the directory of your solution.

You must always submit by Moodle and check *results.txt* there!

The school solution is also available. Once you have a client/program you want to attach our library to and run, use:

```
~os/ex/ex2/run_school_solution <files>
```

Where <files> is a list of files to compile along the school solution (one of them MUST have a main function, of course). You can also give a directory.

Submission

Submit a zip/tar file containing the following:

- README file - The README should be structured according to the course guidelines.
- The headers: MapReduceJob.h, MapContext.h, ReduceContext.h.
- The source files for your implementation of the library.

You **must not** submit the other headers (MapReduceClient.h and MapReduceKeys.h).